

Document#: P4139R0
Date: 2026-03-23
Project: ISO SC22/WG21 Programming Language C++
Title: Better better lookup
Reply-to: ncm@cantrip.org
Target: C++26
Audience: LEWG

Better better lookup

P3091 instituted a member `get` for associative containers, a monadic analog of `op[]` and `at()`.

While of obvious utility, it is the first instance in the library of a `get()` that can fail. It is also the first `get()` that (commonly) invokes a loop; every other instance provides access to an immediately available object at a remove of, at most, a fixed, compile-time offset. When first considered in Sofia, the vote (weakly) favored retaining the name `get`, but discussion since has indicated receptivity to an alternative. This paper is about choosing an alternative.

Discussion

P3091 offered `get_optional`, `lookup`, and `lookup_optional`, which failed to evoke enthusiasm. Interest at the time was focused on semantics, with discussion of names felt by many to be a distraction. The only criterion for a good name mentioned was "short".

The only alternative seriously considered at the time was `lookup`, which was rejected by weak consensus. The goal of this paper is not to advocate for a particular choice, but rather for any practical choice consistent with other usage in the library, and more descriptive and apt than the opaque `get`.

As a reminder, in use, the member would typically appear in a context like

```
if (auto maybe = container.xxxxx(key)) {  
    use(*maybe);  
}
```

or

```
use(container.xxxxx(key).value_or(mapped{}));
```

Proposal

Here I propose `try_at(key)` but would welcome other suggestions consistent with established usage in the Standard. We might consider again `lookup(key)`, and `look(key)`, and variants `try_lookup(key)` and `try_look(key)`.

Reference

[1]: [P3091R3] Pablo Halpern - Better lookups for `map` and `unordered_map` <https://wg21.link/p3091r3>